LiveRecall

Revision-aware live captions with bounded recovery and per-client delivery

Ng De Heng • Foo Juinn Kai • Jay Kim • Ong Cheng Leong

1 System architecture

The supplied deterministic source emits provisional revisions and a final caption for each segment. A WebSocket server validates and distributes that stream to many browser clients. The browser replaces revisions in place, retains recent finals, and provides a local 15/30/60-second recall view.

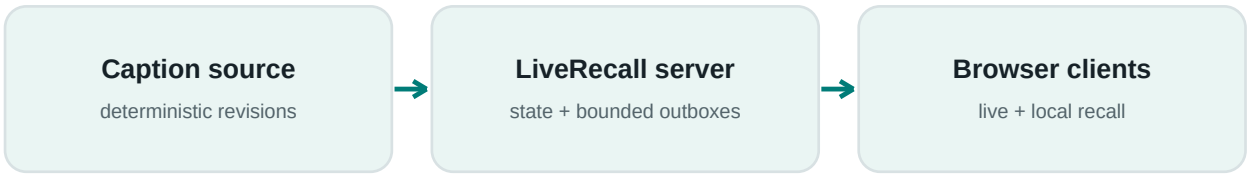

Figure 1. Required data path. The server keeps bounded session history and separate per-client outboxes.

Protocol

- The source sends **SOURCE_HELLO**, then **CAPTION_UPDATE** objects with session, segment, revision, source timestamp, final flag, and text.
- A student sends **CLIENT_HELLO** with the latest finalized segment/revision and a requested history window. The server replies with oldest-first **HISTORY**, then live updates.
- The browser stores one current caption per segment, distinguishes provisional and final text, reconnects automatically, and requests only recent missed finals after a gap.

The same submitted server and browser are used for the deterministic application demo and the measured tests; speech recognition is outside the mandatory path.

2 Correctness and recovery

The server accepts a revision only when it is newer than the stored revision and the segment has not already become final. This prevents stale updates from overwriting completed text. Final captions enter a bounded recent-history deque. On reconnect, the client sends its latest finalized cursor; the server returns only newer finals within the 60-second recovery limit. The browser deduplicates history against local segment state.

A late joiner starts with cursor zero and receives recent context before live updates. Snapshotting history and subscribing a client occur without an intervening await, avoiding a gap or duplicate around the handoff. The browser holds at least 60 seconds of recent finalized captions locally for ordinary recall.

3 Semantic backpressure

Each client has a separate bounded outbox and sender task, so one slow receiver cannot block the source handler or normal readers. Under normal conditions the full revision stream is forwarded. When the network path falls behind, TCP acknowledgement feedback holds provisional revisions while finals proceed. Application-level ping/pong feedback detects a reader that has stopped consuming even when TCP has accepted bytes; that reader is held and then paced near its measured reading rate.

- Unsent obsolete provisional revisions can be replaced with the latest state of their segment; finalized captions remain durable.
- Several pending finals may be bundled in one catch-up HISTORY frame, reducing the number of reads a slow client must perform.
- Per-client pending work is capped; a client that falls too far behind is closed so it can resume through bounded history rather than growing an unlimited queue.

Trade-off

Coalescing lowers provisional update coverage during overload but protects final-caption delivery and freshness. The control loops should return to full-update delivery once lag clears. The browser demo separately verifies revision replacement, local recall, reconnect, and late join.

The six deterministic server correctness checks passed on the Ubuntu VM used for the controlled slow-client study; the public and hidden performance workloads probe additional conditions.

4 Public experimental method

The supplied harness starts a fresh server for each run and logs caption receipt by simulated clients. Tests run on Ubuntu with the source and clients in one VM; T5 applies Linux tc/netem to loopback. Each reported scenario has three independent runs and median metrics. Caption Age is client receive time minus source timestamp. Freshness uses the public 500 ms threshold.

Required public tests

Test	Condition	P50	P95	P99	Fresh	Finals	Compl.	Cover.
T1	20 normal clients	11	30	59	100.0%	100.0%	100.00%	100.00%
T2	100 normal clients	20	41	52	100.0%	100.0%	100.00%	100.00%
T3	100; 10% slow	21	50	3569	98.4%	96.8%	100.00%	94.52%
T4	100; 20% reconnect	19	38	45	100.0%	98.4%	100.00%	98.25%
T5	100; delay + 3% loss	293	818	1247	83.6%	84.6%	97.94%	96.22%

T1-T4 had 100% median final completeness, but minimum individual-run completeness was 95.45%, 97.96%, 100%, and 97.37%, respectively. T3's slow readers raised P99 to 3.57 s; T4 recovery P95 was 15.1 ms. All reported T1-T5 runs had zero connection errors, duplicate captions, and stale captions.

T5 network profiles

Profile	P50	P95	P99	Fresh	Finals	Compl.	Cover.
100 +/- 20 ms delay	222	273	296	100.0%	100.0%	100.00%	100.00%
120 +/- 30 ms delay; 3% loss	293	818	1247	83.6%	84.6%	97.94%	96.22%
3% loss	26	64	274	99.8%	85.7%	85.71%	86.33%

Profiles used Linux tc/netem on loopback: 100 ± 20 ms delay, 120 ± 30 ms delay with 3% IP packet loss, and 3% loss alone. Each used 100 clients, a 40-second window, and three completed runs. Minimum per-run completeness was 100%, 97.82%, and 83.67%. The combined profile's P95 rose to 818 ms; the loss-only profile's 85.71% median completeness is a material shortfall despite its 64 ms P95 among observed captions. TCP retransmissions can increase tail latency; these summaries alone cannot establish why expected finals were absent before the cutoff.

5 Controlled slow-reader comparison

A separate A/B study used 100 clients, 30% slow readers sleeping 500 ms after each message, exactly 76 source segments, and a 100-second client window. Three independent runs were made with the final lag-probe design and with **LIVERECALL_BACKPRESSURE=0**. The fixed segment count and longer window followed a 60-second sweep whose role breakdown exposed the cutoff confound.

Lag probes	Overall P95	Slow P95	Fresh finals	Final completeness	Connection errors
Enabled	1,908 ms	8,321 ms	88.95%	100 / 100 / 100%	0 / 0 / 0
Disabled	35,563 ms	42,356 ms	70.39%	92.5 / 100 / 100%	7 / 30 / 30

With probes enabled, median slow-client P95 was about 34 seconds lower and the fresh-final ratio was 18.56 percentage points higher. Normal-client P95 stayed below 30 ms in both configurations. Normal update coverage was 83.68% with probes and 92.73% without, consistent with replacing obsolete provisional revisions during overload. No duplicate or stale captions were recorded in these runs.

6 Limitations and future work

The A/B result is one controlled workload on one VM with three repetitions; hidden tests may combine larger populations, burstier revisions, constrained bandwidth, and impairment removal. The initial 60-second sweep shows why observation-window timing must be inspected before interpreting missing finals. T5 packet loss exposed a completeness shortfall; per-run source and delivery timestamps are needed to separate source timing, retransmission delay, and server behavior.

Future work could adapt read-rate estimation to other WebSocket client libraries and network paths, measure convergence after a bottleneck clears, and integrate a real lecture speech recognizer while preserving the same protocol and mandatory lightweight server path.

Sources: CS3103 Assignment 4 LiveRecall brief (17 Oct 2026); supplied harness and code; controlled study in docs/slow_client_evaluation.md.